AI ALGORITHM PATTERNS · PATTERN 01

The Top-K Pattern: One Idea, Everywhere in AI

Heaps, priority queues, and the single trick behind search engines, RAG, beam search, and self-driving cars.

Merit Point Academy · 30 min · press **S** notes · → next · 💬 to comment

AGENDA

Where we're going

- **0–6'** Warm-up + what a heap actually is
- **6–14'** Play with a live heap-insertion animation
- **14–20'** The Top-K pattern, then run real Python
- **20–27'** Where Top-K hides: KNN → search → RAG → beam search → self-driving
- **27–30'** Connect it to your own research tool & wrap-up

By the end: explain what a min-heap is, why Top-K beats sorting everything, and name three AI systems secretly running the same pattern.

WARM-UP · ACTIVATE

Quick check · tap an answer

A website computes a similarity score for 1,000,000 topics and needs only the 10 most relevant. What's the smart way to find them?

- Sort all 1,000,000 scores, then take the first 10
- Keep a small structure of size 10 and update it as scores come in
- Ask a human to read all 1,000,000
- It can't be done without a supercomputer

CONCEPT 1 · THE DATA STRUCTURE

What is a heap?

- A **heap** (priority queue) is a tree stored as a flat array where **every parent obeys a rule** versus its children.
- In a **min-heap**: `parent ≤ children` — so the smallest value is always at the root, findable in O(1).
- Insert and remove both cost only **O(log n)** — the tree "sifts" the new value up or down until the rule holds again.
- Python's stdlib `heapq` module implements exactly this, on a plain list.

Array `[1,3,8,5,6]`

↕

Tree

same data, viewed as parent/child pairs at indices `i` and `2i+1`, `2i+2`

Why it matters: a heap lets you always grab "the best (or worst) one so far" cheaply — which is exactly what "keep the Top K" needs.

HANDS-ON · WATCH IT SIFT

Try it: insert values into a min-heap

Insert a value — watch it get added at the end of the array, then **"sift up"**, swapping with its parent until the min-heap rule holds again. Step through with the controls below.

HEAPQ · SIFT-UP ON INSERT

```
def heappush_with_steps(self, val):
    self.heap.append(val)
    self._sift_up(len(self.heap) - 1)

def _sift_up(self, idx):
    while idx > 0:
        parent = (idx - 1) // 2
        if self.heap[parent] > self.heap[idx]:
            self.heap[parent], self.heap[idx] = \
```

Min-heap property: every parent ≤ its children.

INSERT VALUE

e.g., 7 | Insert

QUICK INSERT

5 3 8 1 6 2 9

CURRENT HEAP (ARRAY)

[]

Step: 0 / 0

Action: Ready. Insert a value to see the animation.

HEAP AS A BINARY TREE

Step 0
(empty)

(empty)

[]

⏮ First | ◀ Prev | ▶ Play | Next ▶ | Last ⏭ | Speed

CHECK · AFTER THE ANIMATION

Quick check · tap an answer

You insert a new value at the end of the heap's array. When does it stop 'sifting up'?

- After exactly one swap, always
- When it reaches the root, no matter what
- As soon as its parent is smaller than (or equal to) it
- It never stops — it sifts up every time you insert

CONCEPT 2 · THE PATTERN

The Top-K pattern

The problem shows up constantly: **"out of many candidates, keep only the best K."** Two ways to solve it:

Naive

sort all n items — O(n log n)

then

take the first K

Better

keep a size-K heap

for each item

push it, then pop the worst if size > K — O(n log K)

When **K is small and n is huge** (10 out of a million), O(n log K) crushes O(n log n). This exact problem is **LeetCode 347: Top K Frequent Elements** — same idea, many names.

HANDS-ON LAB · RUN REAL PYTHON

Find the Top-K with Python's heapq

Python's built-in heapq module does exactly this. Run it, then try changing K or writing your own version with heapq.heappush / heappop.

```python
import heapq

# A tiny "search engine": (document name, similarity score)
scores = [
    ("Paper A", 0.92), ("Paper B", 0.74), ("Paper C", 0.98),
    ("Paper D", 0.55), ("Paper E", 0.81), ("Paper F", 0.67),
    ("Paper G", 0.90), ("Paper H", 0.72),
]

K = 3
top_k = heapq.nlargest(K, scores, key=lambda s: s[1])

print(f"Top {K} most relevant results:")
for name, score in top_k:
```

▶ Run | ✨ Explain | 🐞 Debug | ⚡ Improve | Python loads on first Run

```
▶ Run the code to see the output here.
```

✨ AI Assistant — ⚡ Haiku · fastest

Hi! Edit the code and hit **Run**. Stuck? I can **Explain**, **Debug**, or **Improve** it — or just ask me below.

Ask about this code… | Ask

WHERE TOP-K HIDES IN AI · PART 1

Classical ML: rarely, sometimes, sometimes

Classical ML — almost never

Linear/logistic regression, SVMs, CNNs, Transformers mostly do **matrix multiplication** and **gradient descent** — no heap involved.

Tree-based ML — sometimes

Decision trees & random forests don't use heaps. XGBoost/LightGBM use their own clever internal structures, not `heapq`.

KNN — sometimes

Naive: compute all distances, sort, take K. Better: maintain a **size-K max-heap** (or a KD-tree / ball tree) as you scan.

> Not every AI system needs Top-K — but the moment a system has to **rank and shortlist**, this pattern reappears.

WHERE TOP-K HIDES IN AI · PART 2

Search & RAG: Top-K is (almost) the whole job

Score everything, keep only the best K — this **is** retrieval.

User question | **Embedding** | **Similarity score ×50,000** | **Top K documents** | **→ LLM**

This is exactly **LeetCode 347** at scale. Production vector databases like **FAISS, HNSW, ScaNN** use far more advanced structures than `heapq` — but conceptually they're all solving the same thing: *keep only the best K candidates.*

WHERE TOP-K HIDES IN AI · PART 3

Beam search: Top-K inside every token

When an LLM generates text, it doesn't keep just one guess — it keeps the **Top K candidate sequences** at every step.

```
"I love" →
"I love cats" · score 0.91
"I love dogs" · score 0.90
"I love pizza" · score 0.89
…
```

Every decoding step: **expand** each kept sequence with candidate next words, then **keep only the best K** ("the beam width") and drop the rest.

Many beam-search implementations use a priority queue to track the current Top-K sequences — the identical pattern as Lab 1, just running thousands of times per response.

WHERE TOP-K HIDES IN AI · PART 4

Beyond language: self-driving perception

A self-driving car's perception system might detect **300 nearby objects** in one frame. Planning doesn't reason about all 300 equally.

- Keep the **closest 20** objects
- or the **most dangerous 10** (by predicted collision risk)

300 objects → Top 20 relevant
Ranking under time pressure, every frame.

Top K again — same pattern, completely different field.

> Google Search (billions → Top 10), RAG (millions of chunks → Top 20), Waymo (hundreds of objects → Top 20), beam search (thousands of sequences → Top 5) — **all the same computational pattern.**

CONNECT IT · YOUR OWN RESEARCH TOOL

ScholarMind AI runs this pattern too

When ScholarMind AI turns your interests into research topics, it ranks candidate topics/datasets by relevance and shows you only the best few — the same Top-K shape as everything else today:

🔎 Your topic tool

Query → embed → score datasets → **Top K topics**

🌐 Google Search

Billions of pages → **Top 10**

📚 RAG

Millions of chunks → **Top 20**

🚗 Waymo

Hundreds of objects → **Top 20** relevant

💬 Beam search

Thousands of sequences → **Top 5**

Once you can name the pattern, you can spot it everywhere — that's the whole point of this mini-course series.

CHECK · PUTTING IT TOGETHER

Quick check · tap an answer

Beam search, RAG retrieval, and self-driving object ranking all share which underlying idea?

- They all use the same neural network architecture
- They all keep only the best K candidates out of many, usually via a heap
- They are all examples of AGI
- They all require GPUs to run

WRAP-UP

Three things to keep

- A **heap** keeps "the best/worst so far" available in O(log n), without a full sort
- The **Top-K pattern** — score many, keep only K — powers search, RAG, beam search, KNN, and even self-driving perception
- Naming a pattern once means **recognizing it everywhere** — the goal of this whole mini-course series

Exit ticket: name one place in a system you use daily where "keep only the top few" is probably happening. Post it in 💬.

Next pattern · Two Pointers & Sliding Window — how attention scans a sequence.